

General Training Tips

A Practical Checklist for Training Deep Learning Models on Medical Images

Original course slides: June 16, 2024 • Extended into a practical debugging checklist

What This Deck Covers

1

Diagnose Before You Optimize

Sanity checks, LR behavior, train/val curves, logging, baselines

2

Represent Medical Data Deliberately

2D vs 3D vs patch-wise, multi-view, multi-modality fusion

3

Data & Preprocessing Discipline

Normalization, augmentation, imbalance, patient-level splits

4

Loss Functions & Optimization

Loss selection, LR scheduling, weight decay, stability

5

Models, Checkpoints & Ensembles

Checkpointing strategy, prediction averaging, EMA/SWA

6

Debugging Workflow & Final Checklist

Controlled experiments and the 10-step debug order

SECTION 6 · FINAL CHECKLIST

1

Can it overfit 16 samples?

2

Are input/target visualizations correct?

3

Is the loss appropriate?

4

Is the LR reasonable?

5

Is the train/val split clean (no patient leakage)?

6

Is preprocessing consistent?

7

Is the metric implemented correctly?

8

Is the model large enough?

9

Is augmentation too strong?

10

Is the dataset/label quality limiting performance?

Most failures are bugs, data issues, or LR issues — before they are “architecture” problems.

Key Takeaways

- ✓ Sanity-check before scaling up — overfit a tiny subset first.
- ✓ Diagnose with curves and visuals, not intuition.
- ✓ Choose your medical-data representation deliberately (2D / 3D / patch / multi-view / multi-modal).
- ✓ Log images, not just numbers.
- ✓ Ensemble predictions, not weights.
- ✓ For generative/diffusion models, judge quality from samples and image metrics — not the loss curve.
- ✓ Adapt pretrained models efficiently with partial fine-tuning or LoRA/PEFT.
- ✓ Change one variable at a time.

01

Diagnose Before You Optimize

Sanity checks, learning-rate behavior, and reading your curves

Overfit a Tiny Subset First

Take 1, 8, 16, 32, or 64 samples

Train until the model nearly memorizes it.

Expected: loss $\rightarrow \sim 0$, accuracy / Dice / PSNR $\rightarrow$ very high.

- **If it can't overfit a tiny subset, something is probably wrong:**

- bug in labels
- wrong loss
- wrong tensor shape
- bad normalization
- too much augmentation
- learning rate too low / too high
- model output not aligned with target

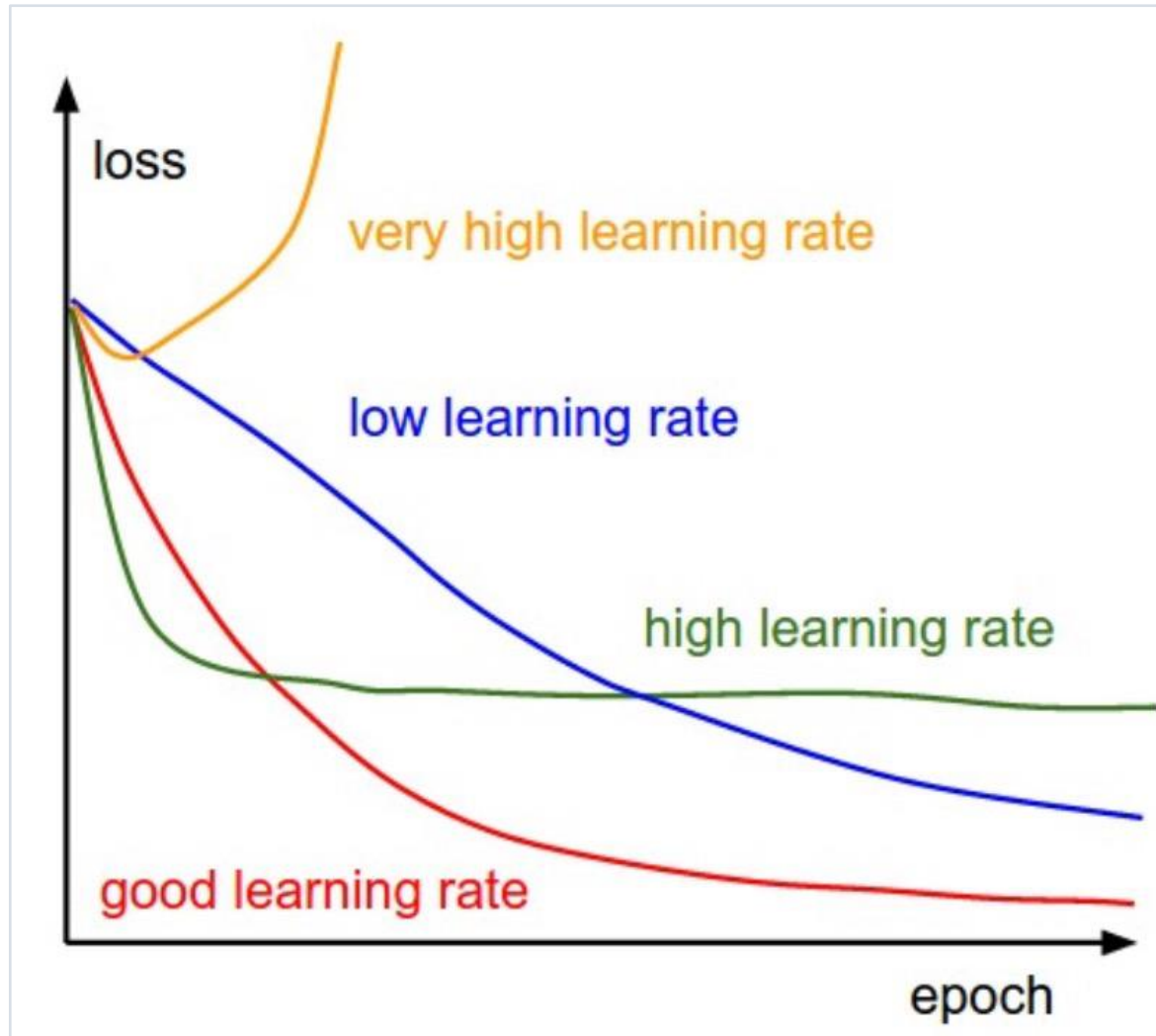
Why this matters most

This is one of the most useful debugging tests available.

It isolates your pipeline (data, loss, model wiring) from the much harder question of whether the model generalizes.

Do this before any serious training run — it takes minutes and catches the majority of “why isn't this learning” bugs.

Reading the Loss Curve



- **Good LR**
 - loss drops quickly, then gradually plateaus — the normal shape
- **Low LR**
 - stable but slow; decreases steadily, wastes training time
- **High LR**
 - drops, then plateaus at a higher, noisy loss
- **Very high LR**
 - loss explodes / diverges

Diffusion Models: Don't Trust the Loss Curve

Training objective: predict the noise added at a randomly sampled timestep t , averaged over t , x_0 , and ϵ each batch — $L = E[t, x_0, \epsilon] \|\epsilon - \epsilon_\vartheta(x_t, t)\|^2$

Why the curve is misleading:

- t is re-sampled randomly every batch — huge batch-to-batch variance that has nothing to do with model quality
- loss magnitude differs a lot across noise levels, so one aggregate epoch average can look flat while specific timesteps clearly improve or worsen
- MSE in (latent) noise space correlates weakly with perceptual sample quality
- loss can plateau — or keep slowly falling — while visual sample quality keeps changing substantially

Not just diffusion

GAN losses (minimax) are famously non-monotonic and don't track sample quality.

VAE ELBO can look healthy while samples stay blurry (posterior collapse).

Across generative models: the training loss is a proxy for the objective you optimized — not for what you actually care about: realistic, diverse, correctly-conditioned samples.

What to Track Instead: Samples & Image Metrics

Visual Monitoring

- sample on a fixed schedule (e.g. every N steps)
- use a FIXED set of seeds/noise so checkpoints are directly comparable
- sample with the real sampler (DDPM/DDIM/ancestral) — not teacher forcing
- sample using EMA weights, not raw weights
- log a grid of samples, not one cherry-picked image

Quantitative Metrics

- FID — distributional similarity to real data (standard)
- Precision / Recall — separates fidelity from diversity
- LPIPS — perceptual similarity / sample diversity
- CLIP score — text-image alignment (text-to-image)
- PSNR / SSIM — for paired tasks: reconstruction, denoising, super-resolution

Also break the loss down by timestep bucket (early / mid / late noise) instead of one aggregate number — it often reveals exactly where the model is struggling.

Symptoms → Fixes

LR Too High

- **Symptoms:**
 - loss becomes nan
 - validation unstable
 - gradients explode
 - training never settles
- **Fix:**
 - lower LR by 2×, 5×, or 10×

LR Too Low

- **Symptoms:**
 - training stable but painfully slow
 - model underfits after many epochs
 - loss curve almost flat
 -
- **Fix:**
 - increase LR by 2× or 3×

Batch Size $\leftrightarrow$ Learning Rate Thumb Rules

Let $k = B' / B$ (new batch size $\div$ old batch size)

SGD / SGD + Momentum — Linear Scaling

- $\eta' = k \cdot \eta$
 - batch size: 32 $\rightarrow$ 128 ($k = 4$)
 - lr: 1e-2 $\rightarrow$ 4e-2
 - double batch size $\rightarrow$ double LR
 - halve batch size $\rightarrow$ halve LR

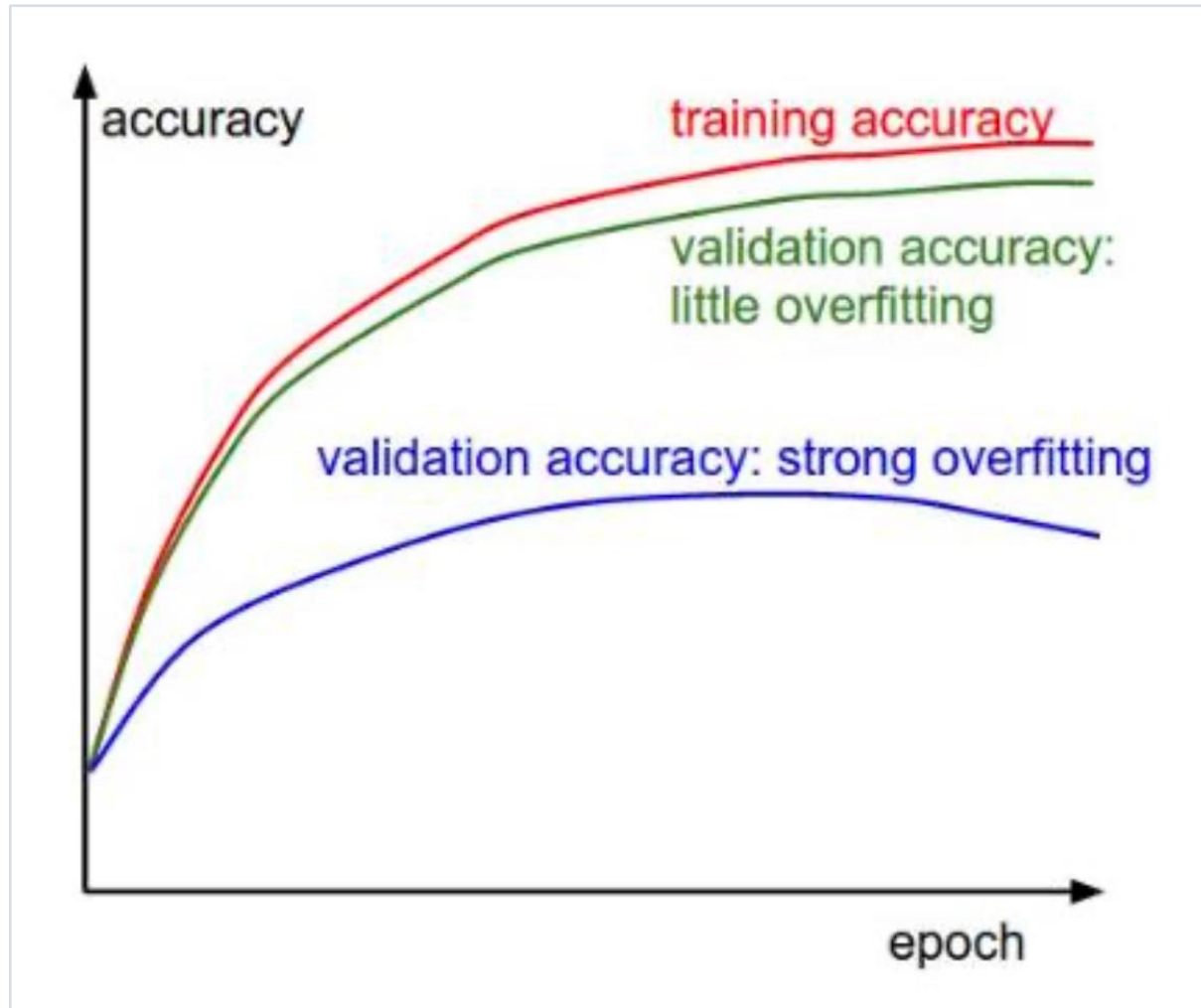
Adam / AdamW — Square-Root Scaling

- $\eta' = \sqrt{k} \cdot \eta$
 - batch size: 32 $\rightarrow$ 128 ($k = 4$)
 - lr: 1e-4 $\rightarrow$ 2e-4
 - double batch $\rightarrow$ LR $\times \sqrt{2}$
 - quadruple batch $\rightarrow$ LR $\times 2$

Effective batch size: $B_{\text{eff}} = B_{\text{per_GPU}} \times N_{\text{GPUs}} \times N_{\text{grad_accumulation}}$

Caveat: for very large batches these rules become too aggressive — use LR warmup.

Use the Curves to Diagnose the Problem



- **Healthy training**
 - train improves, validation improves, small gap
- **Overfitting**
 - train keeps improving, validation stalls or worsens
- **Underfitting**
 - both train and validation are bad

Fixing Overfitting vs. Underfitting

Overfitting

- more augmentation
- stronger weight decay
- dropout
- early stopping
- smaller model
- more data
- less training time

Underfitting

- larger model
- train longer
- higher learning rate
- weaker regularization
- better architecture
- check labels / preprocessing

Log Everything

- Use TensorBoard, Weights & Biases, or CSV logs.

Log at least:

- train loss
- validation loss
- task metric
- learning rate
- gradient norm
- example predictions
- checkpoint path
- random seed & config

For imaging tasks, also log visuals:

- input image
- target image
- prediction
- error map
- mask / segmentation overlay

Numbers alone hide many bugs — always look at images.

Start With a Boring Baseline

Before using a complex method, train a simple one:

UNet

ResNet

ViT baseline

2D CNN

Simple classifier

Zero-filled MRI + CNN

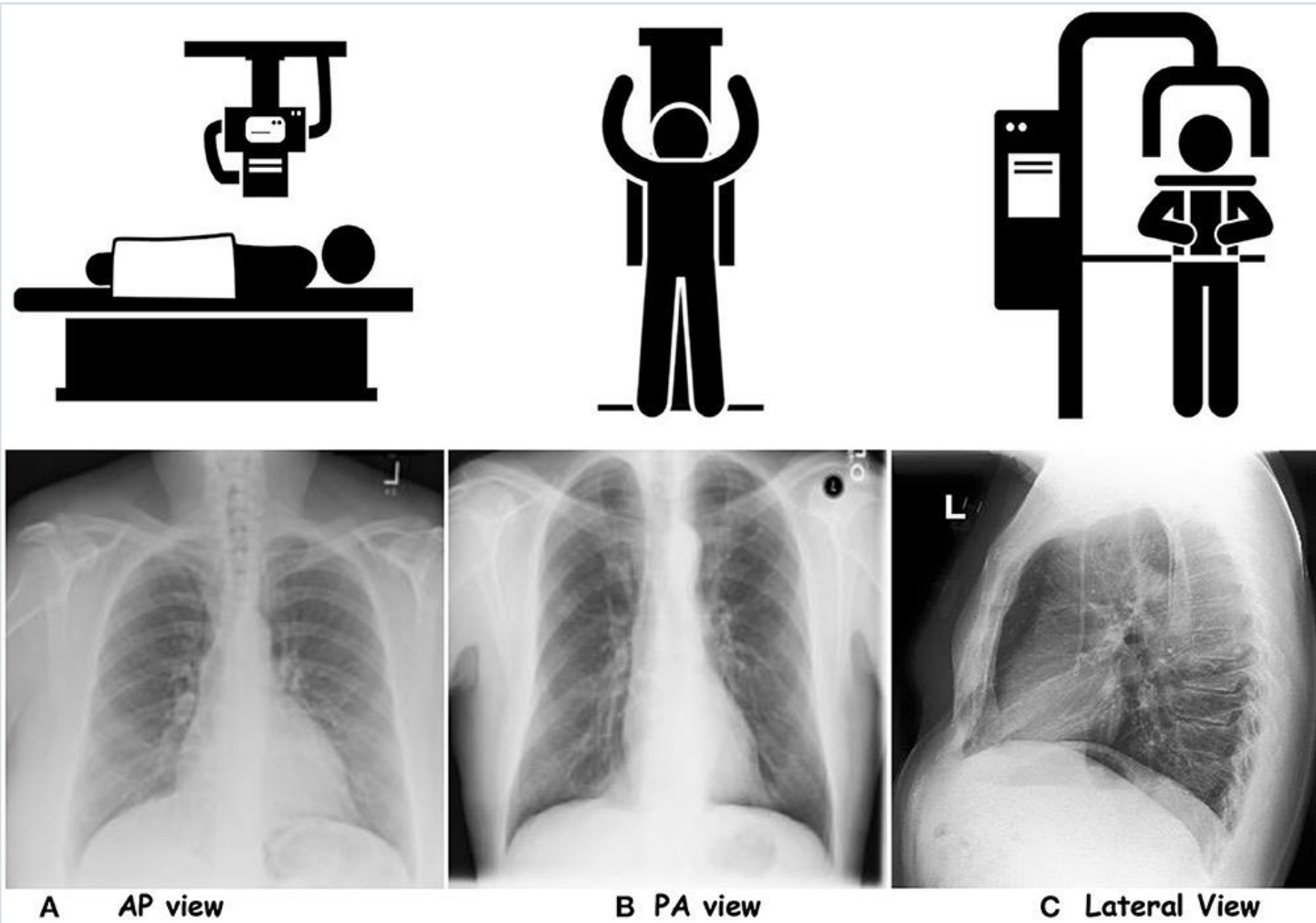
The baseline tells you whether the task is learnable at all — and whether your fancy method is actually helping.

02

Represent Medical Data Deliberately

2D, 3D, patch-wise, multi-view, or multi-modal — the choice matters

Multiple Views Are Available



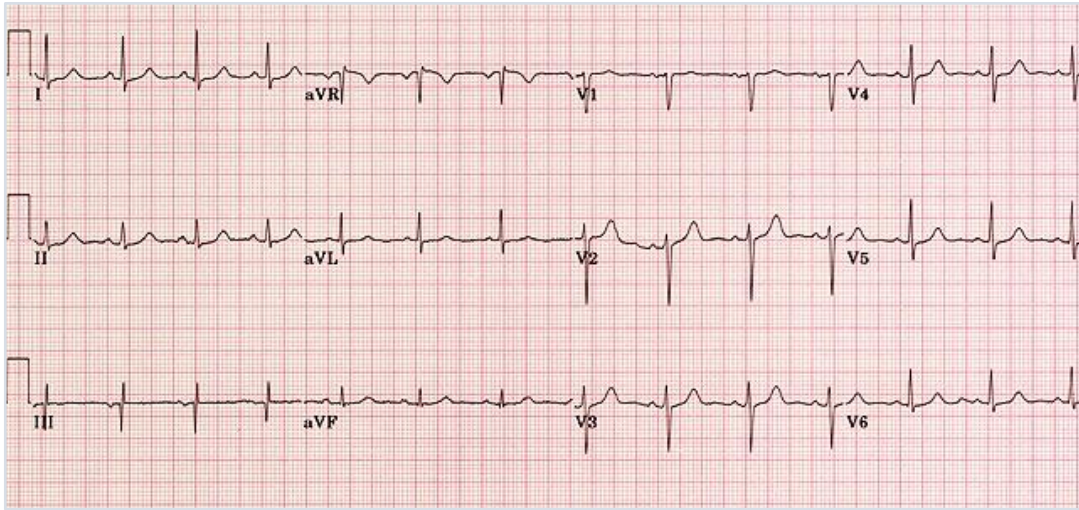
- **X-rays contain 3 views:**

- AP (anterior-posterior)
- PA (posterior-anterior)
- Lateral

Should all views be used?

- single-view model
- multi-view model
- view-specific encoders

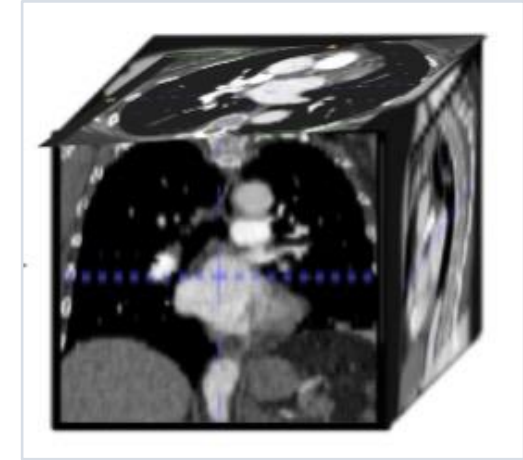
1D, 2D, 3D, or Patch-wise?



ECG: 1D signal or rendered as a 2D image



CT: 2D axial / coronal / sagittal



CT: 3D volume

- ECG can be processed as a 1D signal (1D-CNN / RNN) or as a rendered 2D image.
- CT can be processed as 2D slices, a full 3D volume, or as smaller 3D patches — each with different memory and context trade-offs.

CT Representation Trade-offs

2D Slice

✓ Simpler, less memory

✗ Loses through-plane context

3D Volume

✓ More spatial context

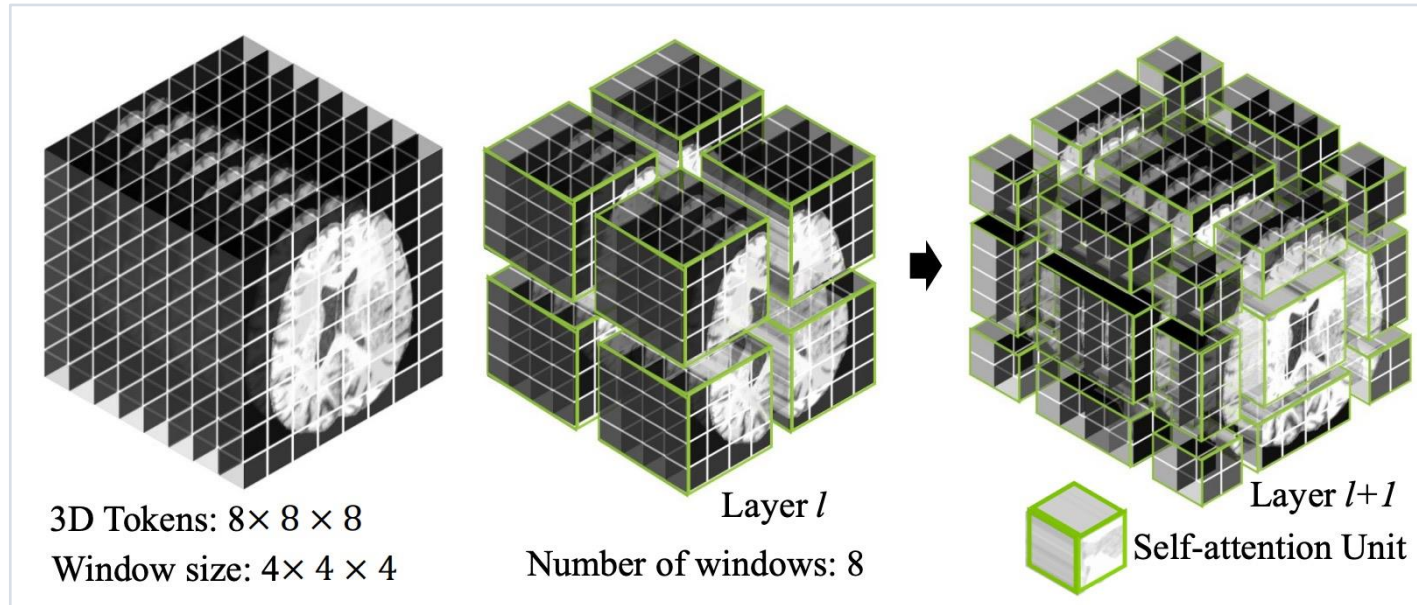
✗ More memory, higher overfitting risk

Patch-wise

✓ Great for small lesions

✗ May lose global context

Patches Help With Small Pathology



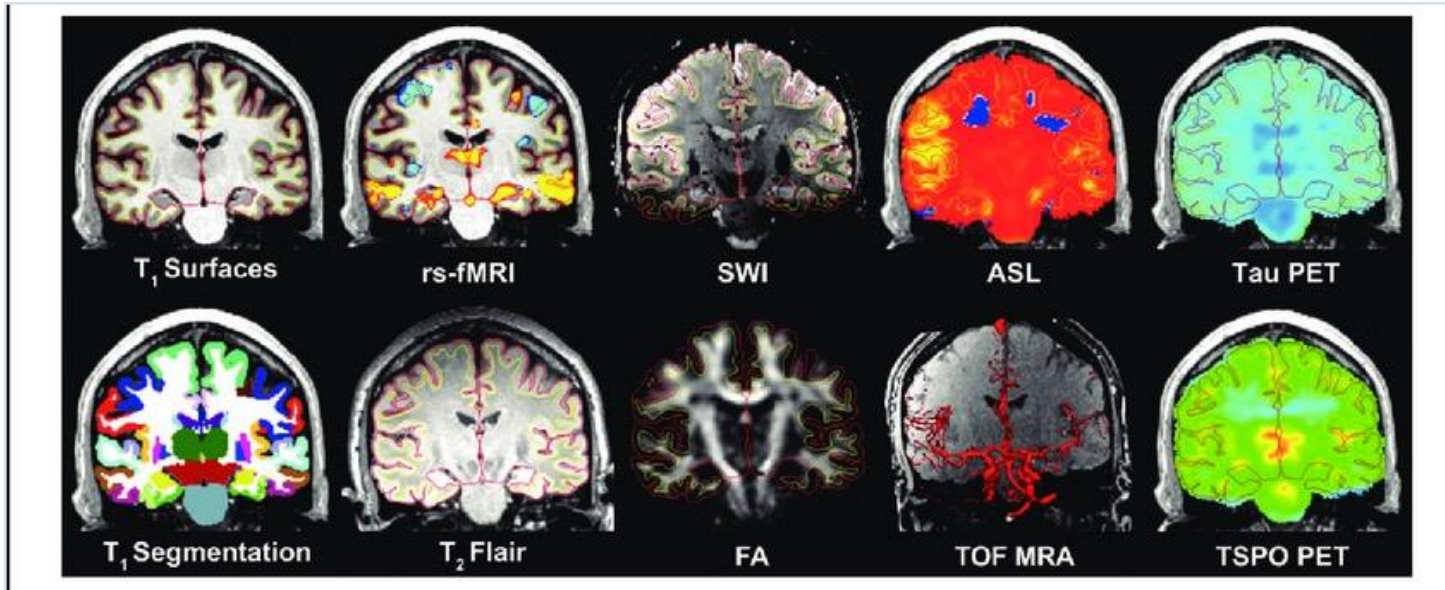
- **For small pathology:**

- high resolution matters
- patches may help preserve detail
- aggressive downsampling can destroy the signal

Examples:

- small lesions
- microbleeds
- tiny tumors
- vessels, calcifications

Multiple Modalities, Multiple Fusion Strategies



T1, T2/FLAIR, SWI, ASL, PET and derived maps — all covering the same anatomy

- **MRI modalities can be:**

- modeled with separate T1/T2/FLAIR models
- early-fused: concatenated as channels
- late-fused: separate encoders, features combined later

Ask:

- should each modality have its own encoder?

How to Use Your Data — Ask First

- Is there a pretrained network for the same task?
- How much data do you have?
- Is the pathology small or large? 2D or 3D in nature?
- What annotations exist (voxel / slice / image-level)?
- 2D slices, 3D volumes, patches, multi-view, or multi-modal?
- Should modalities be concatenated as channels, or use separate encoders?

This choice can matter more than small optimizer tweaks.

Plan all processing alternatives before writing training code — the representation decision shapes your architecture, memory budget, and how much data augmentation is even valid.

Ensemble models usually improve results on top of whichever representation you choose.

Match Architecture to Pathology Size

Small Pathology

- high resolution matters
- patches may help
- aggressive downsampling destroys signal

Examples:

- small lesions, microbleeds
- tiny tumors, vessels
- calcifications

Large / Global Pathology

- larger field of view matters
- global context matters
- lower resolution may be acceptable

Examples:

- organ-level disease
- large tumors, whole-brain atrophy
- lung opacity

03

Data & Preprocessing Discipline

A strong model cannot fix broken preprocessing

Check Preprocessing Before Blaming the Model

Common bugs:

- wrong intensity normalization
- train/val normalization mismatch
- wrong image orientation
- wrong spacing / resolution
- label misalignment
- mask interpolation with bilinear instead of nearest
- augmentations applied incorrectly to targets

Medical-imaging-specific checks:

- voxel spacing
- slice thickness
- orientation
- resampling
- windowing
- modality-specific normalization

A strong model cannot fix broken preprocessing.

Normalize Consistently

Pick one deliberately:

- per-image normalization
- per-volume normalization
- dataset-level mean / std
- percentile clipping
- z-score normalization
- min-max normalization

MRI

Per-volume normalization is common — absolute intensity is often not standardized across scanners.

CT

Hounsfield units have physical meaning — use windowing / clipping carefully rather than generic normalization.

Augment, But Not Blindly

Useful augmentations:

- flips
- small rotations
- scaling / cropping
- intensity jitter, noise, blur
- elastic deformation
- gamma / contrast changes

Is it anatomically valid?

- horizontal flip: fine for brain, risky for laterality-sensitive tasks (heart)
- large rotation: usually invalid for chest X-rays
- intensity jitter: can break CT Hounsfield-unit meaning
- elastic deformation: may distort the pathology itself

Handle Imbalance Explicitly

Options:

- weighted loss
- focal loss
- oversampling rare cases
- balanced batch sampler
- patch sampling around lesions
- threshold tuning
- report per-class metrics

Don't rely only on accuracy:

AUC

F1

Sensitivity

Specificity

Dice

IoU

Balanced accuracy

PR-AUC

Validation for Selection, Test for Final Evaluation

Train

optimize weights

Validation

choose hyperparameters / checkpoints

Test

final unbiased evaluation

Never tune on the test set.

For medical imaging, split by PATIENT — not by slice, image, or patch.

Bad: the same patient appears in both train and validation.

This causes leakage and inflated results — a model can “recognize” a patient's anatomy rather than generalizing.

04

Loss Functions & Optimization

Picking the right loss, scheduling LR, and keeping training stable

Use the Right Loss for the Task

Classification

- CrossEntropyLoss
- BCEWithLogitsLoss
- FocalLoss

Segmentation

- Dice loss
- CE + Dice
- BCE + Dice
- Tversky loss
- Boundary loss

Regression / Reconstruction

- L1
- L2 / MSE
- SSIM loss
- Perceptual loss
- Frequency-domain loss

Rule of thumb

L2 → smoother outputs L1 → sharper, more robust CE / BCE → classification probabilities Dice → region overlap, useful for imbalance

Prefer BCEWithLogitsLoss

For binary or multi-label classification:

✓ Do this

```
loss_fn = torch.nn.BCEWithLogitsLoss()  
loss = loss_fn(logits, target)
```

✗ Not this

```
prob = torch.sigmoid(logits)  
loss = BCE(prob, target)
```

BCEWithLogitsLoss combines the sigmoid and the loss in one numerically-stable operation (using the log-sum-exp trick), avoiding overflow/underflow that a manual sigmoid → BCE pipeline can hit at extreme logit values.

Use LR Scheduling

Common good defaults:

- cosine decay
- step decay
- ReduceLROnPlateau
- one-cycle LR
- linear warmup + cosine decay

Robust default for AdamW:

warmup for 1–5% of training, then cosine decay

Warmup is especially useful for:

- large batch training
- Transformers
- mixed precision
- unstable early training

Use Weight Decay Correctly

Decoupled weight decay (AdamW)

```
torch.optim.AdamW(  
    model.parameters(), lr=lr, weight_decay=wd  
)
```

Typical starting values:

CNNs: 1e-5 to 1e-4
Transformers: 1e-4 to 1e-2

Often exclude from weight decay:

- bias terms
- BatchNorm / LayerNorm weights
- embedding norms

Gradient Clipping & Mixed Precision

Gradient clipping

- useful for RNNs, Transformers, diffusion, GANs, large models, mixed precision

```
torch.nn.utils.clip_grad_norm_(  
    model.parameters(), max_norm=1.0)
```

If clipping constantly activates, your LR may be too high.

Mixed precision

- `torch.cuda.amp.autocast()` + `GradScaler()` — speeds up training, reduces memory

Check for:

- nan loss / underflow
- unstable Dice / CE loss
- bad normalization layers

Keep numerically sensitive ops in float32.

05

Models, Checkpoints & Ensembles

Save the right checkpoints, then average predictions — not weights

Save Checkpoints Intelligently

Save:

- best validation checkpoint
- last checkpoint
- periodic checkpoints

Naming

```
best_val_loss.pt  
best_val_metric.pt  
epoch_050.pt  
last.pt
```

The best-loss checkpoint is not always the best task-metric checkpoint.

- segmentation: best Dice $\neq$ best CE loss
- reconstruction: best PSNR $\neq$ best perceptual quality

Average Predictions for Stronger Results

Use several trained models — even the same model from different epochs — and average their outputs.

Classification

$$p_{\text{ens}}(y|x) = (1/M) \sum p_m(y|x)$$

Segmentation

$$P_{\text{ens}}(x) = (1/M) \sum P_m(x)$$

Regression / Recon.

$$\hat{x}_{\text{ens}} = (1/M) \sum \hat{x}_m$$

Sources of diversity:

- different random seeds
- different folds / architectures
- different checkpoints from the same run

Cheap trick: checkpoint ensembling

Run epoch_80.pt, epoch_90.pt, epoch_100.pt from the same training run and average their predictions — no extra training needed.

Do Not Naively Average Weights

Usually avoid:

$$\theta_{\text{avg}} = (1/M) \sum \theta_m$$

independently trained networks may represent similar functions with incompatible weights

Exceptions:

- EMA — exponential moving average
- SWA — stochastic weight averaging
- same-run checkpoint averaging, sometimes

Prediction averaging is safer.

06

Debugging Workflow & Final Checklist

Change one thing at a time, and check things in the right order

Inspect Predictions Early

After the first few iterations, visualize outputs. Many bugs are visible after one batch.

? Correct shape?

? Aligned with target?

? Masks shifted?

? Classes swapped?

? Output constant?

? Intensity range correct?

Debug Deterministically, Then Scale Up

While Debugging

- `seed_everything(seed)`
- `num_workers = 0`
- `shuffle = False`
- small dataset, single GPU

After Correctness Is Verified

- enable multi-worker loading
- enable DDP
- enable AMP
- enable large-scale training

Keep experiments controlled — change one thing at a time:

```
baseline
+ augmentation
+ scheduler
+ larger model / different loss    (use config files: model, lr, batch_size, loss, seed ...)
```

When a Model Performs Badly, Check in This Order

1

Can it overfit 1-64 samples?

2

Are input/target visualizations correct?

3

Is the loss appropriate?

4

Is the LR reasonable?

5

Is the train/val split clean (no patient leakage)?

6

Is preprocessing consistent?

7

Is the metric implemented correctly?

8

Is the model large enough?

9

Is augmentation too strong?

10

Is the dataset/label quality limiting performance?

Most failures are bugs, data issues, or LR issues — before they are “architecture” problems.

07

Generative Models & Fine-Tuning

Why diffusion loss curves mislead you, and how to adapt pretrained models efficiently

Fine-Tuning: Full, Partial, or Parameter-Efficient

Full Fine-Tuning

- update every parameter
- best with ample task data + compute
- use a small LR (10-100x below pretraining) + warmup
- risk: catastrophic forgetting, overfitting on small data

Partial / Layer Freezing

- freeze early/backbone layers (generic features)
- train only the head / last-k layers
- optionally: gradual (progressive) unfreezing
- layer-wise LR decay: lower LR for earlier layers

LoRA / PEFT

- freeze pretrained weights entirely
- inject trainable low-rank update: $\Delta W = B \cdot A$, rank $r \ll d$
- typically applied to attention Q/K/V/O projections
- merges into weights at inference — zero extra latency
- <1% trainable params; also: adapters, prompt tuning, BitFit

Choosing between them

Full fine-tune → large dataset + big domain shift. Partial freezing (+ layer-wise LR decay) → medium data. LoRA / PEFT → small clinical datasets, limited memory, or many tasks to swap between.

Key Takeaways

- ✓ Sanity-check before scaling up — overfit a tiny subset first.
- ✓ Diagnose with curves and visuals, not intuition.
- ✓ Choose your medical-data representation deliberately (2D / 3D / patch / multi-view / multi-modal).
- ✓ Log images, not just numbers.
- ✓ Ensemble predictions, not weights.
- ✓ For generative/diffusion models, judge quality from samples and image metrics — not the loss curve.
- ✓ Adapt pretrained models efficiently with partial fine-tuning or LoRA/PEFT.
- ✓ Change one variable at a time.

Thank You

Questions?